

RocketSim Agent Workflow and Token-Efficiency Policy

Purpose

RocketSim uses a multi-agent development process because the project is intended to be scientifically trustworthy as well as educational.

The goal is not simply to generate working code. Each new physical model should be independently challenged for:

- physical correctness;
- numerical correctness;
- test quality;
- domain-specific correctness;
- educational consistency.

This process worked very well in Prompt 04. It caught several subtle issues that ordinary code review could easily have missed. However, Prompt 04 also demonstrated that a scientifically rigorous multi-agent workflow can consume usage very rapidly if every agent repeatedly receives large amounts of overlapping context.

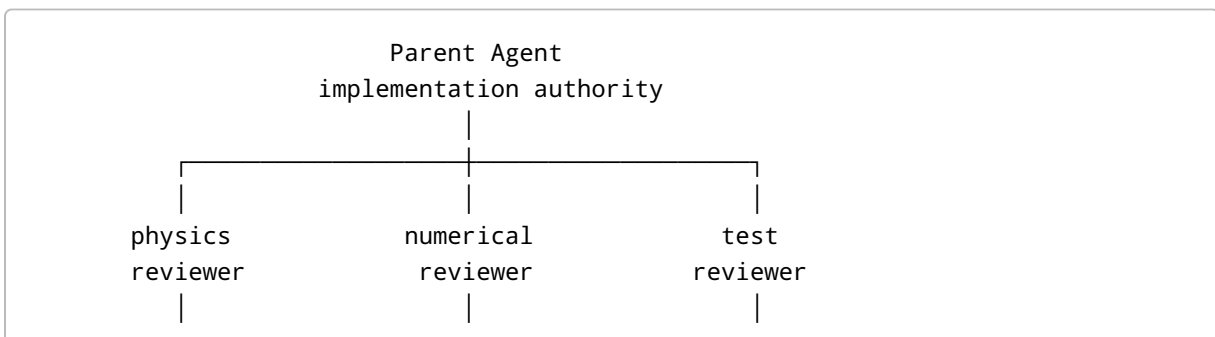
The objective going forward is therefore:

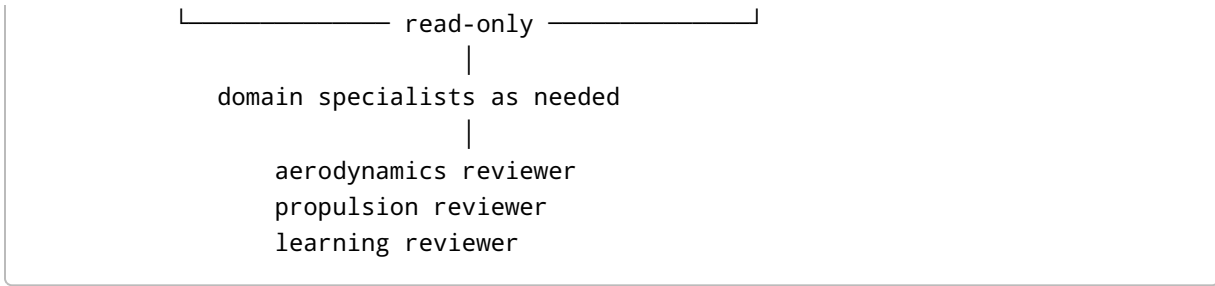
Preserve independent scientific review while giving every agent only the context and work necessary for its role.

1. Parent Agent and Specialist Agents

RocketSim uses one **parent agent** as the sole implementer.

Conceptually:





The parent agent:

- reads the current project state;
- reconciles reviewer findings;
- changes code;
- changes tests and documentation;
- runs validation;
- commits;
- records history;
- pushes.

Specialist agents do **not** implement changes.

They independently inspect the relevant model and report problems to the parent.

This separation is valuable because an implementation agent can unconsciously validate its own assumptions. An independent reviewer is more likely to ask:

“Why is this assumption true?”

rather than:

“Does this code look like what I just wrote?”

2. What the Specialists Do

RocketSim currently has specialist roles such as:

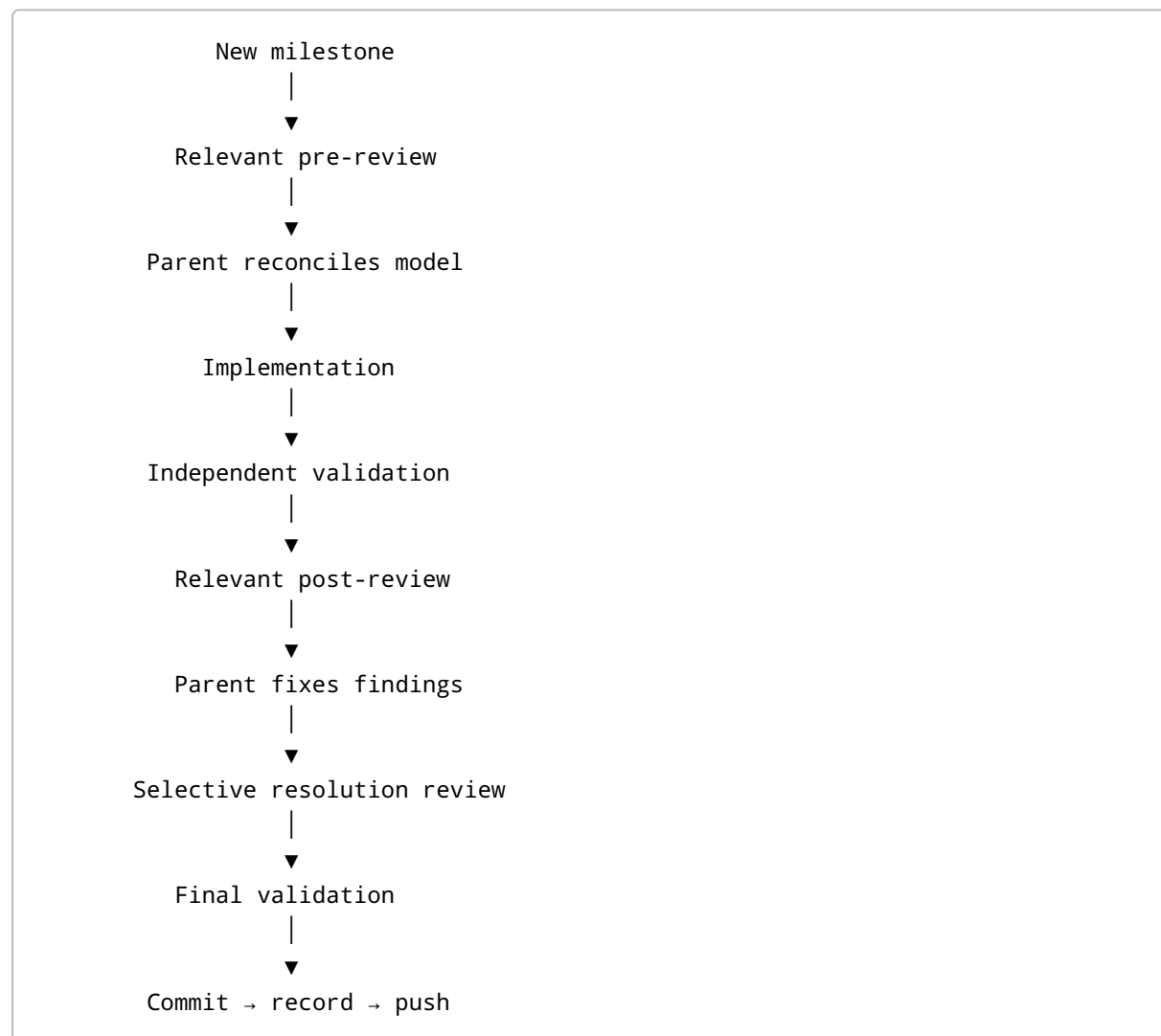
Specialist	Main responsibility
physics_reviewer	equations, forces, signs, physical assumptions
numerical_reviewer	integration, timestep behavior, convergence, event boundaries
test_reviewer	whether incorrect physics could still pass the tests
aerodynamics_reviewer	drag, air-relative velocity, aerodynamic assumptions

Specialist	Main responsibility
<code>propulsion_reviewer</code>	thrust curves, impulse, motor timing and propulsion semantics
<code>learning_reviewer</code>	correctness and pedagogy of the Living Rocketry Course

Prompt 04 demonstrated the value of this structure. The reviewers helped identify or clarify the zero-at-ignition launch problem, powered-impact semantics, thrust-curve endpoint semantics, regression evidence, and several course explanations. The final Prompt 04 record shows all six domains ultimately passed with no remaining BLOCKING or IMPORTANT findings.

3. Normal Review Lifecycle

A scientifically important milestone can follow:



This is intentionally more rigorous than ordinary application development.

For a science-fair simulator, that rigor is useful.

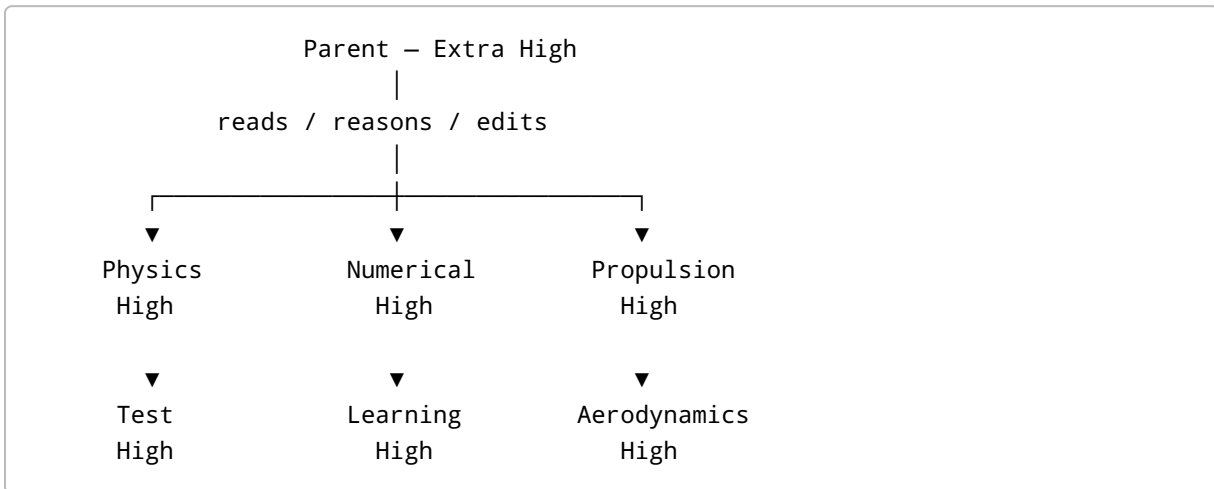
The inefficiency appears when every box causes every specialist to reread most of the entire repository and milestone history.

4. Why Usage Can Fall So Quickly

The Codex “5-hour” allowance should not be interpreted as five hours of wall-clock execution.

A five-minute period can contain many separate model workloads.

For example:



Those specialists may run simultaneously, but they are still separate model computations.

Parallel execution reduces elapsed time.

It does **not** make the work free.

If six reviewers each receive a large prompt, several documentation files, source files, tests, and a large diff, the system may process much of the same information six times.

A simplified example:

100 units of useful project context
×
6 independent reviewers

```
=  
up to ~600 units of context exposure  
  
before considering:  
parent reasoning  
post-review  
resolution review  
documentation  
tool outputs
```

The exact usage accounting is more complicated than this, but the multiplication effect is the important idea.

5. What Made Prompt 04 Especially Expensive

Prompt 04 combined several unusually expensive characteristics.

It had:

```
large implementation prompt  
+  
six specialist domains  
+  
pre-implementation review  
+  
implementation  
+  
post-implementation review  
+  
some resolution re-review  
+  
large source/test diff  
+  
Living Rocketry Course creation  
+  
multiple interruptions/resumes  
+  
large historical prompt record
```

The Prompt 04 archive itself preserves the original large prompt, multiple material resume instructions, reviewer findings, validation evidence, and the exact implementation commit diff.

The existing repository workflow also explicitly requires prompt records to include the implementation commit diff against its parent.

That was excellent for forensic provenance, but it duplicates information Git already stores exactly.

6. Important Distinction: Tools vs. Model Reasoning

Running:

```
pytest
git status
git diff
compileall
```

is not itself the primary source of expensive model reasoning.

The larger cost comes when agents repeatedly:

```
read large context
→ interpret it
→ reason about it
→ produce review findings
→ reread changed context
→ reason again
```

Similarly, a 162-test suite can execute in under a second while a reviewer may spend considerably more model effort analyzing what those tests prove.

The expensive resource is therefore not primarily CPU time.

It is **model context + reasoning + repeated independent review**.

7. New Efficiency Principle

Beginning with the next RocketSim milestone:

Context is a scientific resource. Give each agent the minimum sufficient context required to perform an independent, high-quality review.

“Minimum sufficient” does not mean lowering standards.

It means eliminating irrelevant repetition.

8. Targeted Specialist Context

Future specialist agents should not automatically receive or reread the complete milestone prompt or complete project history.

Instead, each reviewer receives a focused context package.

For example, during a future variable-mass milestone:

```
propulsion_reviewer
  variable-mass equations
  motor/mass-flow decision
  propulsion.py
  relevant simulation code
  relevant tests

physics_reviewer
  governing equations
  mass/thrust/gravity interaction
  relevant physics/simulation code

numerical_reviewer
  integration contract
  timestep/event code
  convergence tests

test_reviewer
  changed model contract
  changed code
  changed/new tests
  independent expected behavior

aerodynamics_reviewer
  only mass/velocity/drag interaction changes

learning_reviewer
  affected course chapters
  authoritative physics changes
  relevant UI behavior
```

A reviewer may request additional context if needed.

The default should no longer be:

“Read everything.”

9. Review Only the Domains That Changed

Not every milestone requires every specialist at full depth.

Future prompts will classify reviewers as:

```
CORE
    directly affected by the new model

FOCUSED
    adjacent domain requiring a narrow regression check

NOT REQUIRED
    unaffected domain protected by existing regression tests
```

For example, variable mass would likely use:

```
CORE
physics
propulsion
numerical
test

FOCUSED
aerodynamics
learning
```

The aerodynamics reviewer would not need to rederive the entire drag model.

It would answer the narrower question:

“Does changing mass alter any established aerodynamic-force semantics or regression behavior?”

10. Selective Re-review

A major Prompt 04 inefficiency was the possibility of repeatedly recalling reviewers after corrections.

Going forward:

A reviewer is re-invoked only when a subsequent change materially affects that reviewer's domain or when its previous BLOCKING/IMPORTANT finding requires confirmation.

Example:

```
learning reviewer requests wording correction
|
▼
course text changes only
|
├─ learning reviewer may recheck
└─ numerical reviewer is NOT rerun
```

Likewise:

```
numerical reviewer requests integrator correction
|
▼
stepping code changes
|
├─ numerical reviewer rechecks
├─ test reviewer may recheck
└─ learning reviewer only if learner-facing behavior changed
```

11. Smaller Parent Prompts

Prompt 04 repeated many policies that now already live in:

```
AGENTS.md
PHYSICS_MODEL.md
VALIDATION.md
ARCHITECTURE.md
```

```
MILESTONES.md
accepted decisions
```

Future prompts should reference those authoritative documents rather than restating them.

Instead of thousands of lines describing established workflow, a future prompt should say:

```
Follow AGENTS.md and all accepted decisions.
```

```
This milestone changes:
```

- X
- Y
- Z

```
Preserve all other validated contracts.
```

```
The following reviewers are relevant:
```

```
...
```

```
The milestone-specific validation is:
```

```
...
```

The repository should increasingly become the durable instruction system.

The prompt should describe only what is **new**.

12. Use Git as the Source of Exact Patch History

The largest archival efficiency improvement is to stop duplicating a complete Git patch inside every Markdown prompt record.

Git already preserves the exact implementation.

Future prompt records should normally store:

```
starting commit SHA
implementation parent SHA
implementation commit SHA
prompt-record commit SHA
zero-context diff SHA-256
exact command used to recover the patch
```

For example:

```
git show --no-ext-diff --binary --format= <implementation-sha>
```

This gives exact reproducibility without copying thousands of lines of patch text into documentation.

The full implementation diff should be embedded only if there is a specific reason to preserve it outside Git.

13. Compact Resume Strategy

Interruptions are unavoidable.

Future resume messages should not restate the entire milestone.

The default resume instruction should be approximately:

```
Resume the approved current milestone from the exact working tree.
```

```
Inspect:
```

- git status
- current diff
- current test state
- completed reviewer results

```
Do not repeat completed reviews.
```

```
Continue from the first incomplete required gate.
```

```
Preserve the reconciled model unless new evidence requires a change.
```

Only new requirements or important unresolved findings need to be repeated.

The repository and working tree should provide the rest of the state.

14. Reasoning-Level Strategy

Use reasoning effort deliberately.

Recommended default:

Parent agent
High

Specialist reviewers
High

Use **Extra High** for the parent when the milestone contains genuinely difficult reconciliation, such as:

- conflicting scientific interpretations;
- major numerical-method decisions;
- new event semantics;
- architecture choices difficult to reverse.

Once those decisions are resolved, return to High for mechanical completion work.

The objective is not to minimize reasoning.

It is to spend the strongest reasoning where it has the highest scientific value.

15. Testing Strategy

Tests remain essential and should not be weakened.

But execution should be staged:

```
during implementation
  focused changed tests

after meaningful integration
  relevant subsystem tests

before specialist post-review
  complete scientific evidence package

before commit
  full suite once

after a material late fix
  affected tests + final full suite
```

Do not repeatedly run every expensive validation step merely because the prompt lists it several times.

A validation result remains valid until code affecting it changes.

16. Historical Reading Strategy

Future agents should normally read:

```
AGENTS.md
current authoritative model docs
current accepted decisions
current implementation
immediately relevant previous prompt record
```

They should not automatically reread every historical prompt from Prompt 01 onward.

Historical records remain available when needed for:

- provenance;
- supersession questions;
- regression archaeology;
- resolving contradictions.

Current authoritative documentation should be the normal working context.

17. Efficiency Must Not Reduce Scientific Independence

Some duplication is intentional and valuable.

For example, a propulsion reviewer independently calculating impulse should **not** simply reuse the implementation's result.

Likewise:

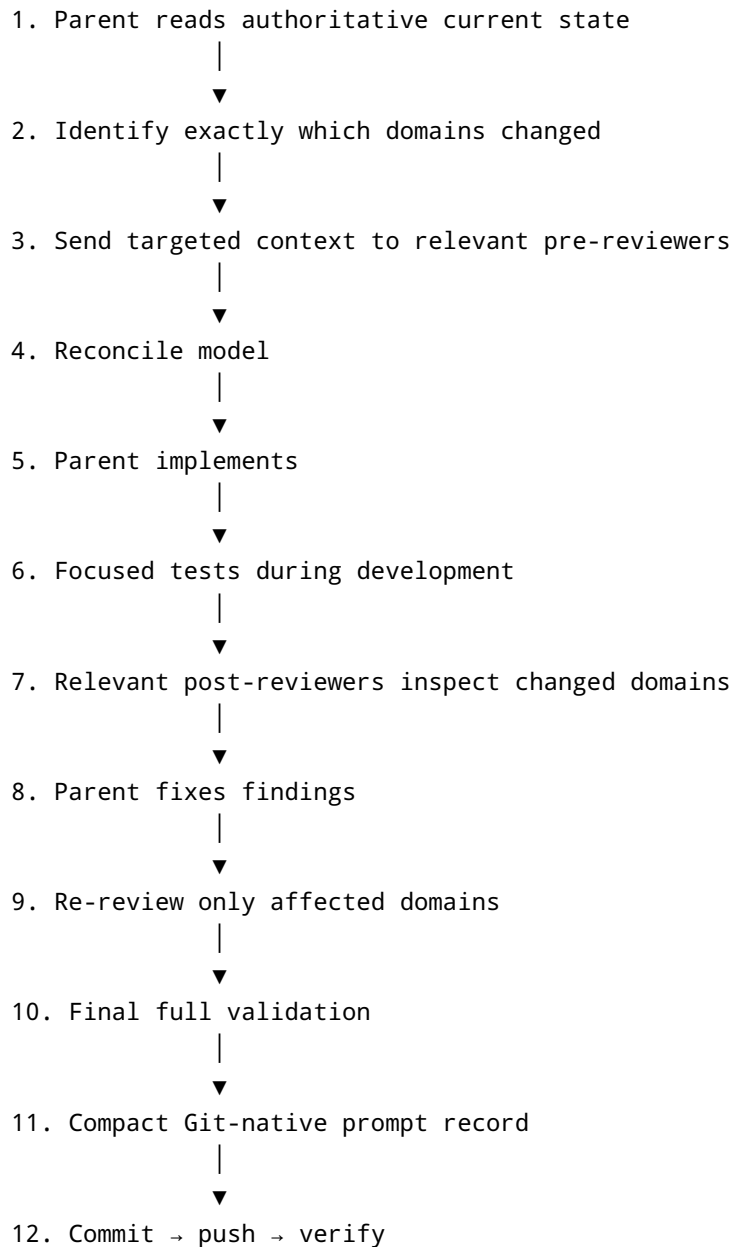
```
production equation
    ≠
test oracle
    ≠
reviewer's independent derivation
```

That duplication is scientifically useful.

The optimization target is **irrelevant context duplication**, not independent reasoning.

18. Revised RocketSim Agent Workflow

The preferred workflow going forward is:



19. What We Keep From Prompt 04

We should **not** abandon:

- independent specialist review;
- analytical validation;
- convergence testing;
- limiting-case tests;
- read-only specialists;
- parent-only implementation authority;
- explicit decision records;
- Living Rocketry Course review;
- regression protection;
- clean implementation/prompt-record commits;
- final push verification.

These produced real value.

20. What Changes After Prompt 04

We will change:

FROM	TO
huge prompts	milestone-specific prompts
all reviewers read everything	targeted reviewer context
all specialists on every change	core + focused review matrix
automatic broad re-review	selective re-review
full history rereading	current authoritative docs
repeated giant resume prompts	compact state-based resumes
embedded full Git patches	SHA + hash + recovery command
Extra High throughout	High normally; Extra High strategically
repeated full validation	staged focused → final validation

21. Standing Policy Going Forward

Beginning with Prompt 05, RocketSim prompt design will follow these rules:

1. **Minimize context without reducing evidence.**
2. **Give specialists only domain-relevant context by default.**
3. **Use core/focused reviewer classification.**
4. **Reinvoke reviewers only when their reviewed domain materially changes.**
5. **Reference durable repository policy rather than repeating it in every prompt.**
6. **Use Git-native provenance instead of embedding complete implementation patches.**
7. **Use short state-based resume instructions after interruptions.**
8. **Read historical prompt records only when they are actually needed.**
9. **Use High reasoning by default; reserve Extra High for genuinely difficult reconciliation.**
10. **Run focused tests iteratively and the complete suite at meaningful gates.**
11. **Preserve independent analytical/test/reviewer derivations even when they duplicate a calculation, because that duplication provides scientific evidence.**
12. **Never trade scientific correctness for token savings.**

The objective is:

More scientific confidence per unit of model context and reasoning.

Not fewer reviewers for the sake of fewer reviewers.

Not shorter documentation for the sake of shorter documentation.

Rather:

Send the right information to the right agent at the right time.

22. Repository Implementation Requirement

This efficiency policy should become durable repository policy rather than depending on conversation memory.

At the beginning of the next approved RocketSim development prompt, Codex should:

```
update AGENTS.md
+
create the next appropriate workflow decision record
```

The decision should codify:

- targeted specialist context;
- core/focused reviewer classification;
- selective re-review;
- compact milestone prompts;
- Git-native prompt provenance;
- compact resume behavior;
- current-authoritative-document reading;
- reasoning-effort strategy;
- staged validation.

The existing Prompt Archive policy should specifically be revised so the implementation record stores:

```
implementation SHA
parent SHA
diff SHA-256
exact git command to reproduce the diff
```

instead of requiring the complete patch to be embedded in the Markdown prompt record by default.

The current repository policy explicitly requires recording the implementation diff, so this needs to be changed deliberately rather than silently ignored.

Conclusion

Prompt 04's high usage was largely the consequence of **high-quality but highly duplicated work**.

Six independent specialists, large context, multiple review rounds, several interruption/resume cycles, and exhaustive archival provenance created a strong result—but also caused the same information to be processed repeatedly.

The multi-agent architecture itself is worth keeping.

The improvement is to change:

```
EVERY AGENT KNOWS EVERYTHING
```

into:

```
THE PARENT KNOWS THE PROJECT
```

EACH SPECIALIST KNOWS
THE PART IT MUST INDEPENDENTLY CHALLENGE

That preserves scientific independence while giving each unit of model reasoning substantially more mileage.